

Specifica del progetto

1. Obiettivo

Il progetto consiste nella realizzazione di un sistema informativo su Web per la gestione di un catalogo di **figure da collezione**, dei **fumetti** e delle **serie** da cui derivano, e delle **collezioni personali** degli utenti registrati.

Il sistema deve essere implementato utilizzando:

- Spring Boot (backend)
- JPA / Hibernate (persistenza)
- PostgreSQL (o altro RDBMS)
- Thymeleaf (frontend)
- React (almeno per una parte del frontend)

L'obiettivo e' progettare e implementare un sistema informativo completo, con particolare attenzione a:

- architettura a livelli
- gestione delle transazioni
- accesso efficiente ai dati
- separazione delle responsabilita'

2. Scenario

Il sistema cataloga le figure da collezione prodotte da diverse aziende. Ogni figure appartiene a una serie (un manga, un anime, un videogioco, un fumetto) e viene prodotta da una sola azienda; la stessa azienda pubblica inoltre serie gacha, cioe' collezioni di figure a sorpresa vendute in capsule o blind box.

Ogni serie contiene uno o piu' personaggi ed e' declinata anche in volumi a fumetti. Un fumetto e' scritto da un solo scrittore ma disegnato da uno o piu' disegnatori, ed e' pubblicato da una casa editrice.

Il sistema distingue tra utenti generici, utenti registrati e utenti amministratori. Gli utenti registrati possono recensire le figure e gestire la propria collezione personale (figure possedute, wishlist, preordini), mentre gli amministratori possono accedere alle funzionalita' di gestione del catalogo.

3. Entita' principali

Serie

Attributi minimi:

- nome
- tipo (MANGA, ANIME, FUMETTO, GRAPHIC_NOVEL, VIDEOGIOCO, FILM, SERIE_TV)
- descrizione
- logo

Relazioni minime:

- una serie contiene uno o piu' personaggi
- una serie raccoglie piu' fumetti
- una serie raccoglie piu' figure

Personaggio**Attributi minimi:**

- nome
- descrizione

Relazioni minime:

- un personaggio appartiene a una serie

Figure**Attributi minimi:**

- nome
- dataUscita
- altezza
- materiale (PVC, ABS, RESINA, POLYSTONE, ...)
- immagine
- prezzo
- descrizione
- edizioneLimitata

Relazioni minime:

- una figure e' prodotta da una sola azienda
- una figure appartiene a una serie
- una figure puo' avere piu' recensioni
- una figure puo' comparire nelle collezioni di piu' utenti

Vincolo di consistenza: la stessa azienda non puo' avere a catalogo due figure con lo stesso nome e la stessa data di uscita.

Azienda**Attributi minimi:**

- nome
- descrizione
- logo

- regione

Relazioni minime:

- un'azienda produce piu' figure
- un'azienda pubblica piu' serie gacha

Gacha

Attributi minimi:

- nome
- descrizione
- dataUscita
- immagine
- prezzo

Relazioni minime:

- una serie gacha e' prodotta da una sola azienda

Fumetto

Attributi minimi:

- titolo
- variant
- anno
- isbn
- prezzo
- genere

Relazioni minime:

- un fumetto appartiene a una serie
- un fumetto e' scritto da un solo scrittore
- un fumetto e' disegnato da uno o piu' disegnatori
- un fumetto e' pubblicato da un solo editore

L'associazione tra Fumetto e Disegnatore e' **molti-a-molti**: un disegnatore lavora su piu' fumetti e un fumetto ha almeno un disegnatore. L'ISBN e' univoco.

Scrittore

Attributi minimi:

- nome
- cognome
- dataNascita
- paese

Relazioni minime:

- uno scrittore puo' aver scritto piu' fumetti

Disegnatore

Attributi minimi:

- nome
- cognome
- dataNascita
- paese

Relazioni minime:

- un disegnatore puo' aver disegnato piu' fumetti

Editore

Attributi minimi:

- nome
- descrizione
- regione
- logo

Relazioni minime:

- un editore pubblica piu' fumetti

Recensione

Attributi minimi:

- titolo
- testo
- voto (1-5)
- data

Relazioni minime:

- una recensione riguarda una figure
- una recensione e' scritta da un utente registrato

Un utente puo' inserire al massimo una recensione per la stessa figure.

VoceCollezione

Attributi minimi:

- stato (POSSEDUTA, WISHLIST, PREORDINE)
- quantita
- dataAggiunta
- note

Relazioni minime:

- una voce collega un utente registrato a una figure

Un utente puo' avere al massimo una voce per la stessa figure.

User / Credentials

Attributi minimi:

- nome, cognome, email (User)
- username, password, ruolo (Credentials)

Relazioni minime:

- a ogni utente corrispondono esattamente delle credenziali di accesso
- un utente puo' scrivere piu' recensioni
- un utente possiede piu' voci di collezione

4. Casi d'uso richiesti

4.1 Funzionalità pubbliche

Il sistema deve consentire almeno:

- visualizzazione del catalogo delle figure
- ricerca delle figure con filtri combinabili (serie, azienda, materiale, fascia di prezzo, edizioni limitate)
- visualizzazione del dettaglio di una figure
- visualizzazione dell'elenco e del dettaglio delle serie
- visualizzazione dei personaggi di una serie
- visualizzazione dei fumetti di una serie
- visualizzazione del dettaglio di un fumetto con scrittore, disegnatori ed editore
- visualizzazione della scheda di uno scrittore o di un disegnatore
- visualizzazione dell'elenco delle aziende e delle serie gacha
- visualizzazione delle recensioni di una figure

Il dettaglio di una serie deve permettere di accedere ai fumetti, alle figure e ai personaggi associati. Il dettaglio di una figure deve mostrare almeno le informazioni sulla figure, l'azienda produttrice, la serie di appartenenza e le recensioni.

4.2 Funzionalità utenti registrati

- inserimento di una recensione relativa a una figure
- modifica di una propria recensione
- eliminazione di una propria recensione
- aggiunta di una figure alla propria collezione (posseduta, wishlist o preordine)
- aggiornamento di una voce della propria collezione
- rimozione di una voce dalla propria collezione
- visualizzazione della propria collezione con il valore complessivo delle figure possedute

Un utente può modificare o eliminare esclusivamente le recensioni di cui è autore e le voci della propria collezione. Non deve essere possibile inserire più di una recensione dello stesso utente per la stessa figure.

4.3 Funzionalità riservate all'amministratore

- inserimento e modifica di una figure a catalogo
- eliminazione di una figure dal catalogo
- inserimento e modifica di un fumetto
- associazione di uno o più disegnatori a un fumetto
- creazione e modifica di una serie
- aggiunta di un personaggio a una serie
- inserimento e modifica di un'azienda produttrice
- inserimento e modifica di scrittori, disegnatori ed editori
- inserimento di una serie gacha

- moderazione: eliminazione di qualsiasi recensione

Il sistema deve verificare la consistenza delle operazioni effettuate. Ad esempio, non deve essere possibile inserire a catalogo due figure con lo stesso nome, della stessa azienda e con la stessa data di uscita, ne' due fumetti con lo stesso ISBN. Non deve essere possibile eliminare una serie, un'azienda o un autore a cui sono ancora collegate altre entita'.

5. Requisiti di sicurezza

L'applicazione deve prevedere autenticazione e autorizzazione, realizzate con Spring Security.

Ruoli

- **DEFAULT**: accesso alle funzionalita' riservate agli utenti registrati (recensioni e collezione)
- **ADMIN**: accesso alle funzionalita' di amministrazione e gestione del catalogo

Requisiti minimi

- login con username e password, password cifrate con BCrypt
- registrazione di nuovi utenti con ruolo DEFAULT
- protezione di tutte le risorse sotto /admin/**
- protezione delle operazioni su recensioni e collezione
- verifica che un utente possa modificare o cancellare esclusivamente le proprie recensioni e le proprie voci di collezione
- protezione CSRF attiva anche per le chiamate provenienti dal frontend React

6. Architettura dell'applicazione

L'applicazione deve essere strutturata secondo una architettura a livelli.

Persistence Layer

- entita' JPA con mapping esplicito delle associazioni e delle strategie di fetch
- repository Spring Data per l'accesso ai dati
- un fragment di repository basato su Criteria API per la ricerca dinamica del catalogo

Service Layer

- implementa i casi d'uso
- coordina uno o piu' repository
- contiene la logica di business e i controlli di consistenza
- gestisce le transazioni
- solleva eccezioni di dominio dedicate

La logica applicativa non deve essere implementata nei Controller.

Controller Layer

- gestisce le richieste HTTP
- valida gli input con Bean Validation
- delega l'esecuzione dei casi d'uso al Service Layer
- prepara i dati destinati al frontend

- traduce le eccezioni di dominio in pagine di errore o codici di stato HTTP

Package	Responsabilita'
model	13 entita' JPA e 3 enumerazioni
repository	repository Spring Data, query JPQL, EntityGraph, fragment Criteria API
service	casi d'uso, regole di business, gestione transazionale
service.exception	EntitaNonTrovata, VincoloViolato, OperazioneNonAutorizzata
controller	controller MVC con viste Thymeleaf
api	controller REST e DTO consumati dal frontend React
security	configurazione di Spring Security
bootstrap	popolamento del database e script di analisi delle strategie di fetch

7. Gestione delle transazioni

I metodi del Service Layer devono essere opportunamente annotati con `@Transactional`, distinguendo tra operazioni di sola lettura e operazioni di aggiornamento. La scelta del livello di isolamento deve essere coerente con il caso d'uso.

Linea guida adottata: le operazioni di sola lettura usano `@Transactional(readOnly = true)`; le operazioni in cui la decisione di scrivere dipende dall'esito di una lettura precedente usano `Isolation.SERIALIZABLE`, perche' con il Read Committed di PostgreSQL due richieste concorrenti potrebbero superare entrambe il controllo.

Operazione multi-entita' di riferimento

Inserimento a catalogo di una nuova figure (`FigureService.inserisciACatalogo`).
L'operazione coinvolge tre repository e si articola in:

- 1 recupero dell'Azienda produttrice
- 2 recupero della Serie di appartenenza
- 3 verifica del vincolo di consistenza (nome + azienda + data di uscita)
- 4 creazione della Figure
- 5 aggiornamento delle associazioni coinvolte

L'operazione deve essere atomica: in caso di errore in un qualsiasi passo, lo stato del sistema deve rimanere consistente e nessuna scrittura parziale deve essere persistita.

Altre operazioni transazionali significative

Operazione	Isolamento	Motivazione
Inserimento recensione	SERIALIZABLE	La scrittura dipende dal controllo «l'utente non ha gia' recensito questa figure».
Aggiunta alla collezione	SERIALIZABLE	La scrittura dipende dal controllo di unicità della coppia (utente, figure).
Inserimento fumetto	SERIALIZABLE	Coinvolge 5 repository e dipende dal controllo di unicità dell'ISBN.

Operazione	Isolamento	Motivazione
Registrazione utente	SERIALIZABLE	Dipende dai controlli di unicità su username ed email.
Lecture di catalogo	readOnly = true	Nessuna scrittura: si evita il dirty checking e si consentono ottimizzazioni.

I vincoli di unicità sono inoltre dichiarati a livello di schema (uk_figure_nome_azienza_data, uk_recensione_autore_figure, uk_voce_utente_figure) come ultima difesa a livello di database.

8. Prestazioni e accesso ai dati

8.1 Progettazione

Le strategie di fetch sono state decise caso per caso in base ai casi d'uso:

Associazione	Fetch	Motivazione
Figure → Azienda	LAZY	Nel catalogo si mostrano molte figure: con EAGER si avrebbe una query per riga. Le query di elenco usano JOIN FETCH esplicito.
Figure → Serie	LAZY	Stessa motivazione: caricata con fetch join dove serve.
Figure → Recensioni	LAZY	Collezione voluminosa, serve solo nella pagina di dettaglio.
Fumetto → Scrittore / Editore / Serie	LAZY	L'elenco dei fumetti usa una query con tre fetch join.
Fumetto ↔ Disegnatori	LAZY	Collezione multi-a-molti: caricata solo nel dettaglio, con left join fetch.
Personaggio → Serie	EAGER	Oggetto singolo sempre mostrato insieme al personaggio.
Gacha → Azienda	EAGER	Oggetto singolo sempre mostrato insieme al gacha.
Recensione → Autore	EAGER	Il nome dell'autore compare sempre accanto alla recensione.
Credentials → User	EAGER	Serve a ogni richiesta autenticata; cascade ALL perché è una composizione.

Le associazioni che esprimono una composizione (Serie→Personaggio, Figure→Recensione, Credentials→User) propagano PERSIST, MERGE e REMOVE; le altre no, per non produrre cancellazioni indesiderate.

La proprietà `spring.jpa.open-in-view` è impostata a `false`: le associazioni necessarie alla vista devono essere caricate esplicitamente nel Service Layer, il che rende visibile e discutibile ogni scelta di fetch.

8.2 Analisi sperimentale dell'accesso ai dati

Il progetto include uno script eseguibile che permette di osservare il comportamento di JPA/Hibernate nell'accesso ai dati. Il caso d'uso analizzato è il **caricamento di tutte le figure di una serie con la relativa azienda produttrice**.

Lo script confronta tre strategie di accesso sullo stesso insieme di dati (il database di esempio contiene 120 figure, 15 per serie):

- 1 **LAZY**: query derivata sull'elenco, associazioni caricate su richiesta
- 2 **JOIN FETCH**: fetch join esplicito nella query JPQL
- 3 **ENTITY GRAPH**: @NamedEntityGraph dichiarativo sull'entita' Figure

Per ciascuna strategia vengono stampati il numero di query SQL eseguite (rilevato tramite le statistiche di Hibernate), il tempo complessivo di esecuzione e il numero di oggetti caricati.

Esecuzione:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=fetch-demo
```

Output atteso:

```
=== Test accesso alle figure della serie "Attack on Titan" ===
```

Strategia 1: LAZY

```
Figure caricate : 15
Query SQL       : 16
Tempo           : 84 ms
```

Strategia 2: JOIN FETCH

```
Figure caricate : 15
Query SQL       : 1
Tempo           : 21 ms
```

Strategia 3: ENTITY GRAPH

```
Figure caricate : 15
Query SQL       : 1
Tempo           : 19 ms
```

Durante la prova orale deve essere possibile eseguire lo script, spiegare le query generate da Hibernate, individuare il problema delle N+1 query, discutere le differenze tra le strategie e prevedere come cambierebbe il comportamento al crescere del numero di entita'.

9. Frontend

L'interfaccia e' realizzata principalmente con Thymeleaf. La parte realizzata in React e' il **catalogo delle figure con ricerca e filtri**, raggiungibile all'indirizzo /catalogo.

La scelta e' motivata: e' la sezione in cui l'interattivita' porta un vantaggio reale rispetto a Thymeleaf, perche' i filtri si combinano e si applicano senza ricaricare la pagina. Il componente React implementa:

- ricerca testuale su nome della figure, serie e azienda, con debounce
- filtri combinabili per serie, azienda, materiale, fascia di prezzo ed edizioni limitate
- ordinamento dei risultati (nome, prezzo crescente/decrescente, uscite recenti)
- pannello di dettaglio con le recensioni della figure selezionata
- gestione degli stati di caricamento e di errore

Il componente e' montato su un div della pagina Thymeleaf e ottiene tutti i dati dal backend attraverso le API REST; non accede mai direttamente al database.

10. API REST

Gli endpoint rispettano le convenzioni HTTP e restituiscono codici di stato appropriati.

Metodo	Endpoint	Accesso	Esito
GET	/api/figure?q=&serieId=&aziendaId=&materiale=&soloLimitate=&prezzoMin=&prezzoMax=	pubblico	200
GET	/api/figure/{id}	pubblico	200 / 404
GET	/api/serie	pubblico	200
GET	/api/aziende	pubblico	200
GET	/api/materiali	pubblico	200
GET	/api/figure/{id}/recensioni	pubblico	200
GET	/api/figure/{id}/recensioni/statistiche	pubblico	200
POST	/api/figure/{id}/recensioni	autenticato	201 / 400 / 409
PUT	/api/recensioni/{id}	solo autore	200 / 400 / 403
DELETE	/api/recensioni/{id}	autore o ADMIN	204 / 403 / 404

Le risposte usano DTO (record Java) e non le entita' JPA: questo evita i cicli di serializzazione delle associazioni bidirezionali, evita le LazyInitializationException fuori dalla transazione e disaccoppia il contratto dell'API dal modello di persistenza.

Gli errori sono restituiti con un corpo uniforme: 400 per input non valido, 403 quando l'utente non e' proprietario della risorsa, 404 per risorsa inesistente, 409 per violazione di una regola di business.

11. Requisiti minimi

Requisito	Come e' soddisfatto
almeno 6 entita' significative	13 entita' JPA

Requisito	Come e' soddisfatto
relazioni uno-a-molti e molti-a-uno	Serie→Personaggio, Azienda→Figure, Serie→Fumetto, Figure→Recensione, ...
almeno una relazione molti-a-molti	Fumetto ↔ Disegnatore, con tabella di join fumetto_disegnatore
relazione uno-a-uno	Credentials ↔ User
almeno 2 casi d'uso per categoria	10 pubblici, 7 per utenti registrati, 10 per amministratori
almeno un caso d'uso multi-entita'	Inserimento a catalogo di una figure (3 repository)
autenticazione e autorizzazione	Spring Security, form login, BCrypt, ruoli DEFAULT/ADMIN
gestione delle transazioni	@Transactional nel Service Layer, isolamento motivato caso per caso
parte del frontend in React	Catalogo con ricerca e filtri su /catalogo
almeno una API REST usata da React	Otto endpoint sotto /api
validazione degli input	Bean Validation su entita' e DTO, BindingResult nei controller
gestione degli errori	Eccezioni di dominio tradotte in pagine di errore e codici HTTP

12. Criteri di valutazione

- correttezza funzionale
- qualita' del modello dati
- corretto mapping delle associazioni JPA
- uso corretto di JPA / Hibernate
- qualita' del Service Layer
- corretta separazione delle responsabilita'
- gestione delle transazioni
- gestione della sicurezza
- qualita' delle API REST
- integrazione tra React e Spring Boot
- efficienza dell'accesso ai dati
- comprensione delle strategie di fetch
- qualita' del codice
- chiarezza dell'interfaccia utente
- validazione dei dati
- gestione degli errori

13. Bonus facoltativi

Elementi gia' presenti:

- ricerca e filtri combinabili sul catalogo
- ricerca dei fumetti per titolo e delle serie per nome
- statistiche sulle recensioni (voto medio e numero per figure)

- valore complessivo della collezione di un utente
- profilo di esecuzione dedicato all'analisi delle strategie di fetch

Elementi previsti come estensione:

- paginazione dell'elenco figure e delle recensioni
- upload di immagini delle figure (attualmente si salva un URL)
- documentazione OpenAPI/Swagger delle API REST
- autenticazione con OAuth
- deployment su una piattaforma cloud

14. Prova orale

Durante la prova orale lo studente deve saper dimostrare di conoscere il progetto e di comprenderne le principali scelte architetture e implementative, ed essere in grado di modificarlo o estenderlo su richiesta. Le modifiche potranno riguardare, ad esempio:

- aggiunta di un attributo o di una nuova relazione tra entità
- modifica di un mapping JPA
- implementazione di una nuova query
- aggiunta di un caso d'uso
- modifica di una regola di business
- aggiunta o modifica di un metodo del Service Layer
- modifica della gestione transazionale
- aggiunta di un endpoint REST
- modifica della configurazione di Spring Security
- modifica della strategia di fetch
- individuazione e correzione di un problema N+1
- modifica di una componente React

Estensioni naturali del modello che possono essere richieste: collegare le figure ai singoli personaggi (oggi il collegamento è alla serie), collegare le serie gacha a una serie, aggiungere una tracciatura dei prezzi nel tempo.

15. Consegna

Inviare un messaggio di posta elettronica all'indirizzo **siw.roma3@gmail.com** entro le ore 12:00 del 13 settembre 2026.

L'oggetto del messaggio deve iniziare con la stringa **[Settembre 2026 PROGETTO PERSONALE]** seguita da cognome e matricola dello studente.

Nel corpo del messaggio riportare:

- indirizzo GitHub (o altra piattaforma Git) del progetto con il codice sorgente
- eventuali malfunzionamenti noti, ma non risolti
- considerazioni generali sull'esperienza